

DOCUMENTACION DE WANDA

WANDA PADEL CLUB - SISTEMA DE RESERVAS



django

Autores:

- Kevin Velasco de Lucas
- David Cruz Mollo
- José Manuel Perdices Hernández
- Rodrigo Romero Escribano

Proyecto: Sistema web de reservas de pistas de padel

Tecnología principal: Python 3.12 y Django 6.0.1

1. INTRODUCCION.....	2
2. OBJETIVOS DEL PROYECTO.....	2
3. TECNOLOGIAS UTILIZADAS.....	3
4. USO DE DJANGO EN EL PROYECTO.....	3
4.1. Arquitectura MTV.....	3
4.2. Proyecto y aplicacion.....	4
4.3. Configuracion en settings.py.....	4
4.4. Enrutamiento con urls.py.....	4
4.5. Modelos y ORM.....	5
4.6. Migraciones.....	5
4.7. Formularios.....	6
4.8. Sistema de autentificacion.....	6
4.9. Decoradores y permisos.....	6
4.10. Plantillas Django.....	7
4.11. Mensajes al usuario.....	7
4.12. Validaciones en Django.....	7
4.13. Transacciones.....	8
4.14. Panel de administracion de Django.....	8
4.15. Tests con Django.....	8
5. ESTRUCTURA DEL PROYECTO.....	9
6. MODELOS DE DATOS.....	10
6.1. Usuario.....	10
6.2. Bono.....	10
6.3. Pista.....	10
6.4. Reserva.....	11
7. FUNCIONALIDADES PRINCIPALES.....	11
7.1. Registro e inicio de sesion.....	11
7.2. Recuperacion de contraseña.....	11
7.3. Compra de bonos.....	12
7.4. Reserva de pistas.....	12
7.5. Anulacion de reservas.....	12
7.6. Home de usuario.....	12
7.7. Panel de administracion personalizado.....	12
8. URLS PRINCIPALES.....	13
9. USUARIOS DE PRUEBA.....	13
10. INSTALACION Y EJECUCION MANUAL.....	14
11. EJECUCION AUTOMATICA.....	14
12. VALIDACIONES Y CONTROL DE ERRORES.....	14
13. TESTS.....	15
14. DECISIONES DE DISEÑO.....	15
15. LIMITACIONES.....	16
16. CONCLUSION.....	16

1. INTRODUCCION

WANDA Padel Club es una aplicacion web desarrollada con Django para gestionar reservas de pistas de padel. El objetivo del proyecto es simular el funcionamiento basico de un club deportivo donde los usuarios pueden registrarse, comprar bonos de créditos, reservar pistas disponibles y consultar su historial.

La aplicacion tambien incluye una zona de administracion personalizada para gestionar usuarios, pistas, bonos y reservas. Ademas, utiliza el sistema de autentificacion de Django, validaciones de modelo, mensajes en pantalla, recuperacion de contrasena y pruebas automaticas.

2. OBJETIVOS DEL PROYECTO

Los objetivos principales del proyecto son:

- Permitir el registro e inicio de sesion de usuarios.
- Diferenciar usuarios normales y administradores mediante el campo is_staff.
- Gestionar pistas de padel desde un panel propio.
- Permitir la compra de bonos de créditos.
- Reservar pistas usando créditos disponibles.
- Evitar reservas duplicadas para la misma pista, fecha y horario.
- Impedir reservas en horarios pasados.
- Permitir la anulacion de reservas futuras devolviendo el crédito.
- Mostrar proximas reservas e historial de reservas finalizadas.
- Registrar el historial de bonos comprados y créditos consumidos.
- Proteger las acciones de administracion para usuarios autorizados.

3. TECNOLOGIAS UTILIZADAS

- Backend:
 - Python 3.12
 - Django 6.0.1
- Base de datos:
 - SQLite3
- Frontend:
 - Django Templates
 - HTML
 - Bootstrap 5
- Autenticacion:
 - Django Auth
 - Modelo de usuario personalizado
- Email:
 - Console Email Backend de Django para desarrollo
- Automatizacion:
 - Script BAT para Windows
 - Script SH para Linux

4. USO DE DJANGO EN EL PROYECTO

Django es el framework principal utilizado para desarrollar la aplicacion. Este proyecto aprovecha varias partes importantes de Django, por lo que no se limita a mostrar páginas HTML, sino que utiliza una estructura completa de aplicacion web.

4.1. Arquitectura MTV

Django utiliza el patrón MTV, que significa Model, Template y View. Este patrón es parecido al modelo MVC, pero adaptado a la forma de trabajar de Django.

En este proyecto:

- Model: se encuentra principalmente en reservas/models.py.
- Template: se encuentra en reservas/templates/reservas/.
- View: se encuentra en reservas/views.py.

Los modelos definen la estructura de los datos, las vistas procesan las peticiones del usuario y las plantillas generan la respuesta HTML que se muestra en el navegador.

4.2. Proyecto y aplicacion

El proyecto Django se llama core. Dentro de core se encuentra la configuracion general, como settings.py y urls.py.

La aplicacion principal se llama reservas. Esta aplicacion contiene la logica del sistema de reservas: modelos, vistas, formularios, plantillas, tests y migraciones.

Esta separacion es una practica habitual en Django, ya que permite organizar el código en bloques funcionales.

4.3. Configuracion en settings.py

El archivo core/settings.py contiene la configuracion principal del proyecto. Algunos elementos importantes son:

- `INSTALLED_APPS`: indica las aplicaciones activas del proyecto.
- `DATABASES`: configura la base de datos SQLite.
- `TEMPLATES`: configura el sistema de plantillas.
- `AUTH_USER_MODEL`: indica que se usa un modelo de usuario personalizado.
- `EMAIL_BACKEND`: configura el backend de email para desarrollo.

Gracias a esta configuracion, Django sabe que aplicaciones cargar, que base de datos utilizar y cómo localizar las plantillas HTML.

4.4. Enrutamiento con urls.py

El archivo core/urls.py define las rutas principales del proyecto. Cada ruta asocia una URL con una vista concreta.

Ejemplos:

- `/` se asocia con la vista home.
- `/registro/` se asocia con la vista registro.
- `/comprar-bono/` se asocia con la vista comprar_bono.
- `/panel-admin/` se asocia con la vista panel_admin.
- `/admin/` carga el panel de administracion propio de Django.

Este sistema permite separar claramente la direccion que escribe el usuario en el navegador de la funcion Python que se ejecuta internamente.

4.5. Modelos y ORM

Los modelos de Django permiten representar tablas de la base de datos usando clases de Python. En este proyecto se usan los modelos Usuario, Bono, Pista y Reserva.

Django ORM permite hacer consultas sin escribir SQL manualmente. Por ejemplo, se pueden filtrar reservas por usuario, obtener pistas activas, crear bonos o actualizar créditos usando metodos de Python.

Ejemplos de operaciones utilizadas:

- **filter()**
- **get()**
- **create()**
- **update()**
- **order_by()**
- **select_related()**
- **select_for_update()**

El ORM facilita el trabajo con la base de datos y reduce errores al no tener que escribir consultas SQL directamente.

4.6. Migraciones

Django usa migraciones para transformar los modelos de Python en tablas reales de base de datos.

En este proyecto, las migraciones se encuentran en `reservas/migrations/`. Cada vez que se cambia un modelo, se puede crear una migracion con:

- `python manage.py makemigrations`

Y aplicar los cambios en la base de datos con:

- `python manage.py migrate`

Las migraciones permiten que el proyecto pueda instalarse en otro ordenador y recrear la estructura correcta de la base de datos.

4.7. Formularios

El proyecto utiliza formularios de Django para gestionar datos enviados por el

usuario. El archivo **reservas/forms.py** contiene formularios como RegistroForm y PistaForm.

Los formularios ayudan a validar los datos, mostrar campos en las plantillas y guardar informacion de manera controlada.

4.8. Sistema de autenticacion

Django incluye un sistema de autenticacion ya preparado. Este proyecto lo usa para:

- Registrar usuarios.
- Iniciar sesion.
- Cerrar sesion.
- Recuperar contrasena.
- Diferenciar usuarios normales y administradores.

El modelo Usuario hereda de AbstractUser, por lo que mantiene las ventajas del sistema de usuarios de Django y al mismo tiempo permite anadir el campo créditos.

4.9. Decoradores y permisos

El proyecto utiliza decoradores para proteger las vistas.

El decorador `login_required` impide que usuarios no autenticados accedan a determinadas funciones, como reservar o comprar bonos.

También se ha creado un decorador propio llamado **admin_required**. Este decorador comprueba si el usuario tiene **is_staff=True** antes de permitirle entrar al panel de administracion personalizado.

4.10. Plantillas Django

Las plantillas se encuentran en `reservas/templates/reservas/`. Utilizan la sintaxis de templates de Django para mostrar datos dinamicos.

Ejemplos de elementos usados:

- Variables: `{{ pista.nombre }}`
- Condiciones: `{% if user.is_authenticated %}`
- Bucles: `{% for reserva in reservas %}`
- Herencia de plantillas: `{% extends 'reservas/base.html' %}`
- Bloques de contenido: `{% block content %}`

Gracias a esto, una misma plantilla puede mostrar información diferente según el usuario, las reservas existentes o el estado de cada pista.

4.11. Mensajes al usuario

Se utiliza `django.contrib.messages` para mostrar avisos en pantalla. Por ejemplo:

- Reserva confirmada.
- Créditos insuficientes.
- Reserva anulada.
- No tienes permisos de administrador.

Esto mejora la experiencia de usuario porque cada acción importante devuelve una respuesta visual.

4.12. Validaciones en Django

El proyecto usa validaciones tanto en vistas como en modelos.

La validación más importante está en el método **`clean()`** del modelo `Reserva`. Este método comprueba que no se pueda reservar en el pasado ni duplicar una reserva existente.

También se usa **`full_clean()`** antes de guardar una reserva, para asegurarse de que las validaciones del modelo se ejecutan antes de insertar datos en la base de datos.

4.13. Transacciones

En algunas operaciones se usa **`transaction.atomic()`** para asegurar que varios cambios se realizan como una sola operación.

Por ejemplo, al reservar una pista se crea la reserva y se descuenta un crédito.

Si algo falla, la transaccion evita que una parte se guarde y la otra no.

También se usa **select_for_update()** en operaciones sensibles de créditos para evitar problemas si dos peticiones modifican el mismo saldo a la vez.

4.14. Panel de administracion de Django

Ademas del panel personalizado, el proyecto conserva el panel de administracion de Django en **/admin/**.

Este panel permite gestionar modelos desde una interfaz generada automaticamente por Django. Es util para tareas internas, revision de datos y comprobaciones durante el desarrollo.

4.15. Tests con Django

Django incluye un sistema de pruebas integrado. En este proyecto se usa `reservas/tests.py` para comprobar comportamientos importantes de la aplicacion.

Los tests se ejecutan con:

- **python manage.py test**

Durante la ejecucion, Django crea una base de datos de prueba separada, ejecuta los tests y despues la elimina. Esto permite comprobar la aplicacion sin danar los datos reales.

5. ESTRUCTURA DEL PROYECTO

La estructura principal del proyecto es la siguiente:

- `proyecto_padel/`
 - `core/`
 - `settings.py`
 - `urls.py`

- asgi.py
 - wsgi.py
- reservas/
 - admin.py
 - apps.py
 - forms.py
 - models.py
 - tests.py
 - views.py
 - migrations/
 - templates/
 - reservas/
- scripts/
 - ejecutar_windows.bat
 - ejecutar_linux.sh
- db.sqlite3
- manage.py
- requirements.txt
- README.md

La carpeta core contiene la configuracion general del proyecto Django.
 La carpeta reservas contiene la aplicacion principal con modelos, vistas, formularios, tests y plantillas HTML.
 La carpeta scripts contiene los archivos de ejecucion automatica.

6. MODELOS DE DATOS

6.1. Usuario

El modelo Usuario hereda de AbstractUser y añade un campo créditos.
 Este campo indica el saldo disponible para realizar reservas.

Campos principales:

- username
- password
- email
- is_staff
- créditos

6.2. Bono

El modelo Bono registra compras o ajustes de créditos asociados a un usuario. Permite saber cuantos créditos tenía inicialmente el bono y cuantos quedan.

Campos principales:

- usuario
- nombre
- créditos
- credits_restantes
- precio
- fecha_compra

También incluye la propiedad credits_consumidos, que calcula la diferencia entre créditos totales y créditos restantes.

6.3. Pista

El modelo Pista representa cada pista de padel disponible en el club.

Campos principales:

- nombre
- imagen_url
- activa

Incluye la propiedad esta_llena_hoy, que comprueba si la pista ya tiene todas las reservas del día ocupadas.

6.4. Reserva

El modelo Reserva representa una reserva realizada por un usuario.

Campos principales:

- usuario
- pista
- bono_consumido

- fecha
- bloque

Los horarios disponibles son:

- 09:00 - 10:30
- 10:30 - 12:00
- 12:00 - 13:30
- 17:00 - 18:30
- 18:30 - 20:00
- 20:00 - 21:30

El modelo válida que no se pueda reservar en un horario pasado y que no existan dos reservas iguales para la misma pista, fecha y bloque horario.

Ademas, se define una restriccion unica en base de datos para reforzar esta regla.

7. FUNCIONALIDADES PRINCIPALES

7.1. Registro e inicio de sesion

Los usuarios pueden crear una cuenta nueva, iniciar sesion y cerrar sesion. El registro utiliza un formulario propio basado en el modelo Usuario.

7.2. Recuperacion de contraseña

La recuperacion de contrasena usa las vistas integradas de Django. El proyecto está configurado con Console Email Backend, por lo que el enlace de recuperacion no se envia por correo real, sino que aparece en la terminal donde se ejecuta el servidor.

7.3. Compra de bonos

Los usuarios pueden comprar bonos de 5, 10 o 15 créditos. Al comprar un bono, Se crea un registro Bono y se aumenta el saldo de créditos del usuario.

7.4. Reserva de pistas

Para reservar una pista, el usuario debe tener al menos un crédito disponible. Al confirmar la reserva, se consume un crédito del bono mas antiguo con saldo y se descuenta también del saldo general del usuario.

7.5. Anulacion de reservas

El usuario puede anular reservas futuras. Cuando se anula una reserva, se devuelve un crédito al usuario y al bono correspondiente.

No se permite anular reservas que ya han pasado.

7.6. Home de usuario

La página principal muestra las pistas disponibles, el estado de cada pista y las reservas del usuario. Las reservas se separan entre proximas reservas e historial de reservas finalizadas.

7.7. Panel de administracion personalizado

Los usuarios administradores pueden acceder a /panel-admin/. Desde este panel se pueden gestionar:

- Usuarios
- Créditos
- Bonos
- Pistas
- Reservas

Las acciones de administracion están protegidas mediante un decorador propio `admin_required`, que comprueba si el usuario tiene permisos de staff.

8. URLS PRINCIPALES

- Página principal:
 - `http://127.0.0.1:8000/`
- Registro:

- <http://127.0.0.1:8000/registro/>
- Login:
 - <http://127.0.0.1:8000/login/>
- Compra de bonos:
 - <http://127.0.0.1:8000/comprar-bono/>
- Panel de administracion personalizado:
 - <http://127.0.0.1:8000/panel-admin/>
- Panel de administracion de Django:
 - <http://127.0.0.1:8000/admin/>

9. USUARIOS DE PRUEBA

- Administrador:
 - Usuario: alumno
 - Contraseña: alumno
 - URL recomendada: <http://127.0.0.1:8000/panel-admin/>
- Usuario normal:
 - Usuario: alumno1
 - Contraseña: alumno1
 - URL recomendada: <http://127.0.0.1:8000/>

10. INSTALACION Y EJECUCION MANUAL

- 1. Crear el entorno virtual:
 - `python -m venv env`
- 2. Activar el entorno virtual en Windows:
 - `.\env\Scripts\activate`
- 3. Activar el entorno virtual en Linux o macOS:
 - `source env/bin/activate`
- 4. Instalar dependencias:
 - `pip install -r requirements.txt`

- 5. Aplicar migraciones:
 - `python manage.py migrate`
- 6. Arrancar el servidor:
 - `python manage.py runserver`
- 7. Abrir la aplicacion en el navegador:
 - `http://127.0.0.1:8000/`

11. EJECUCION AUTOMATICA

El proyecto incluye scripts para facilitar la ejecucion.

- En Windows:
 - `scripts\ejecutar_windows.bat`
- En Linux:
 - `chmod +x scripts/ejecutar_linux.sh`
 - `./scripts/ejecutar_linux.sh`

Estos scripts crean el entorno virtual si no existe, instalan dependencias, aplican migraciones y arrancan el servidor.

12. VALIDACIONES Y CONTROL DE ERRORES

El proyecto incluye varias validaciones importantes:

- No se puede reservar una pista sin créditos disponibles.
- No se puede reservar en un horario pasado.
- No se puede duplicar una reserva para la misma pista, fecha y horario.
- No se puede anular una reserva ya pasada.
- No se puede acceder al panel de administracion sin permisos.
- Los ajustes de créditos se realizan dentro de una transaccion.

Estas validaciones se aplican tanto desde vistas como desde el modelo Reserva, reforzadas con una restriccion unica en base de datos.

13. TESTS

El proyecto incluye tests automaticos ejecutables con:

- `python manage.py test`

Los tests comprueban:

- Reserva duplicada.

- Reserva en el pasado.
- Consumo de crédito al reservar.
- Devolución de crédito al anular.
- Acceso bloqueado al panel admin para usuarios normales.
- Acceso permitido al panel admin para usuarios staff.

14. DECISIONES DE DISEÑO

Se ha utilizado SQLite porque es suficiente para un proyecto academico y no requiere configurar un servidor de base de datos externo.

Se ha usado Bootstrap 5 para crear una interfaz sencilla, clara y responsive.

El sistema de créditos se ha separado en dos partes:

- Saldo general del usuario.
- Historial de bonos comprados.

De esta forma se puede consultar el saldo total y también auditar de donde vienen los créditos.

El panel de administracion personalizado se ha creado para demostrar gestion propia de la aplicacion, ademas del panel admin que Django ofrece por defecto.

15. LIMITACIONES

La recuperacion de contraseña no envia correos reales porque usa Console Email Backend. Esto es correcto para desarrollo y pruebas locales.

Las imagenes de las pistas se cargan mediante URL externa. Si no hay conexion a internet o la URL falla, la aplicacion muestra un texto alternativo.

El proyecto esta preparado para uso local y academico, no para despliegue en produccion sin ajustes adicionales de seguridad, configuracion de email, variables de entorno y base de datos.

16. CONCLUSION

WANDA Padel Club cumple con el objetivo de crear una aplicacion web funcional para gestionar reservas de pistas de padel. El proyecto incluye autenticacion, roles, gestion de pistas, sistema de bonos, reservas con validaciones, historial, panel de administracion personalizado y tests automaticos.

La aplicacion está documentada y puede ejecutarse tanto manualmente como con scripts automaticos, facilitando su revision y prueba en otros ordenadores.